Predictive AI Evaluation Challenge: Cold-Start Probability Modeling with Adaptive Calibration

Pardis Miri
CS321M Predictive Evaluation Challenge
Stanford University
parism

Abstract

This report describes my submission to the CS321M Predictive AI Evaluation Challenge. The task is to predict the probability that a known AI subject answers a cold-start benchmark item correctly, given only subject metadata, item text, benchmark, and condition. I explored smoothed empirical priors, embedding heads, item-response factor models, factor/prior ensembles, adaptive labeling, and Modal-trained variants. The best hidden Codabench score was -0.61 negative log-loss under username `parism`. The strongest family was not the largest neural model; it was a conservative smoothed-prior router with adaptive calibration. Heavier factor models, larger encoders, and learned residual probability models consistently overfit hidden data.

1 Problem Formulation

The challenge asks for a function

$$f(s, i, b, c) = \Pr(Y = 1 \mid s, i, b, c),$$

where s is the AI subject, i is the item text, b is the benchmark, c is the test condition, and Y is binary correctness. The hidden evaluation is item cold-start: test items have no public responses, while subjects mostly appear in the public matrix. The primary metric is mean log-likelihood, reported by Codabench as negative log-loss where higher is better. AUC-ROC is secondary and mainly diagnostic.

The competition also provides an adaptive-labeling channel. A submission may include `labeling.py`, whose `acquisition_function(input)` scores candidate inputs. The platform reveals the top $K = 5$ labels per data category and passes them into `predict(input, labeled)`. This makes calibration as important as ranking: a useful submission must return well-calibrated probabilities, and it must use a tiny number of revealed labels without overfitting them.

2 Data and Validation

I used the public Hugging Face dataset `aims-foundations/measurement-db`. The repository code follows the starter-kit data-loading pattern: response parquet files are listed explicitly, registry tables are loaded separately, and trace tables are excluded. The joined runtime view contains

`benchmark`, `condition`, `subject_content`, `item_content`, `label`.

Only binary correctness labels in $\{0, 1\}$ are used. The active public dataset contains roughly 3.4M binary rows, about 900 subjects, 150k unique items, 16 benchmarks, and 223 benchmark-condition cells.

Preprint.

For local validation I used whole-benchmark and benchmark-aware holdouts rather than random row splits. Random row validation leaks item and benchmark distributional structure and gives overly optimistic estimates. I also built an offline adaptive-label simulator, `simulate_adaptive_labels.py`, which mimics the Codabench loop by selecting the top 5 acquired labels per benchmark-condition category and then scoring the remaining rows with the labeled context.

3 Methods

Engineering baseline and defensive runtime. Before modeling, I fixed several submission-level issues that materially affected runtime reliability. The item text distribution is heavy-tailed: the median item has about 300 characters, but the longest public item is about 25k characters. Batching those texts without truncation made sentence embedding slow because one long item pads the whole batch. I therefore truncated item strings consistently during training and inference and capped the encoder sequence length. I also fixed a Codabench-specific NumPy 2 issue: locally, `float()` on a one-element two-dimensional array only warned, but on Codabench it raised a `TypeError`. The final wrappers use explicit scalar extraction, string-normalized keys, finite-probability clipping, and broad fallback paths so a single malformed input cannot crash a submission.

Smoothed-prior baseline. The simplest useful model estimates empirical correctness rates at several granularities and shrinks each cell toward a parent mean. The final prior uses the hierarchy

$$\text{subject-benchmark-condition} \to \text{subject-benchmark} \to \text{benchmark-condition} \to \text{subject/benchmark} \to \text{global}.$$

The first baseline averaged several cells and scored about -0.64 hidden log-loss. A stronger v2 baseline preferred the deepest populated cell, especially subject-benchmark-condition, with Bayesian shrinkage toward coarser cells. This improved local NLL dramatically, but hidden Codabench revealed that the finest cells were less stable under distribution shift than local validation suggested. The final robust prior router therefore blended older and newer prior families instead of trusting the deepest cell unconditionally.

Embedding head. I trained a content-based head using SentenceTransformer item embeddings plus smoothed subject priors. The first version used MiniLM item embeddings and `class_weight=balanced`; it had strong in-sample diagnostics (NLL about 0.557, AUC about 0.788) but hidden score only -0.64, indicating calibration and cold-start mismatch. I then removed class balancing and added a subject embedding interaction, concatenating item embedding, subject prior, and the subject's mean embedding profile. This made the smoke-test prediction much better calibrated to the public base rate, but hidden performance dropped to -0.69. The lesson was that raw text embeddings and subject-conditioned item averages overfit public benchmark structure.

Factor/PGE model. I implemented a three-stage prediction-guided-evaluation pipeline: first fit a rank-1 item-response/factor model with subject ability θ_s and item parameters (a_i, b_i); then train an MLP from MiniLM item embeddings to item parameters; finally predict

$$\sigma((a_i\theta_s - b_i)/T)$$

for cold-start items. The first factor model trained on 500k rows and scored -0.61 with AUC 0.70, the first submission with clear discriminative signal. I then expanded to roughly 1.9M rows, a two-layer 256-unit MLP, theta weight decay, and temperature calibration. Local held-out NLL improved sharply after fitting $T = 0.275$, but hidden performance regressed in several calibrated variants. Multi-seed MiniLM ensembling, MPNet/BGE-style larger-encoder experiments on Modal, and bigger MLPs reduced some validation losses but did not beat the simpler robust prior family on hidden Codabench.

Factor/prior ensembles and robust prior router. The factor model and smoothed priors made different mistakes, so I built logit-space ensembles. A global 70/30 factor/prior ensemble tied the -0.61 range, but per-benchmark routing based on local diagnostics overfit badly: benchmarks where the public baseline looked dominant did not map cleanly to hidden evaluation. The best final family was a robust prior-only router between the older v1 weighted prior and the deeper v2 prior. Without labels, it uses a conservative default blend. With revealed labels, it compares the v1 and v2 priors on labeled examples for the current category and softly shifts the logit-space blend toward the better prior. A separate clipped logit blend shrinks predictions toward the observed mean of the revealed labels for that category.

Adaptive acquisition. The acquisition function must never fail: any exception or non-finite score triggers random fallback for the entire round. I tried length-biased acquisition, uniform stable hashing, uncertainty/entropy, v1/v2 disagreement, diversity tiers, and category-aware variants. Length bias was harmful because it selected long code/math items whose revealed labels were systematically harder than the rest of the category; applying those residuals globally pulled many predictions down. The most robust submissions use simple deterministic uncertainty or hash selection and clipped adaptive calibration.

4 Experiments and Ablations

Table 1 summarizes the main Codabench results, ordered from lower to higher hidden negative log-loss score. The best score was -0.61, reached by several conservative prior-family submissions. Larger or more aggressive variants generally worsened hidden log-loss. Appendix A gives the fuller iteration log from the development transcript.

Submission	Score (neg. log-loss)	AUC	Interpretation
`embedding v2`	-0.69	–	Subject-embedding interaction overfit cold-start distribution.
`robust_mpnet`	-0.65	–	Larger encoder/factor artifact did not generalize.
`baseline` / `embedding v1`	-0.64	–	Early prior and class-balanced MiniLM head plateaued.
`robust_prior_v2_w60`	-0.63	–	Higher v2 weight looked good locally but overfit hidden data.
`robust_prior_clip/hash/diverse`	-0.62	–	Acquisition and label-blend ablations.
`factor_baseline_ensemble`	-0.62–-0.61	0.69	Helpful when conservative; per-category routing overfit.
`factor_pge v1`	-0.61	0.70	2-PL factor model gave the strongest AUC signal.
`robust_direct_residual_w15`	-0.61	–	Tiny learned residual tied best; larger residual weights hurt.
`robust_prior_v1/v2`	-0.61	0.68	Conservative prior router; best leaderboard entry.

Table 1: Selected Codabench results, ordered from lower to higher hidden Codabench negative log-loss score. Higher is better because scores are negative. The best `parism` entry was -0.61 with AUC 0.68.

The main ablation pattern was consistent. First, calibration and shrinkage mattered more than model complexity. The factor model had useful discriminative signal on some benchmarks, but it was poorly calibrated on others and was often dominated by smoothed priors. Second, adaptive labels helped only when used gently. Removing the label blend hurt, but stronger correction and smaller clips could also hurt. Third, local simulation was useful for screening variants, but it was not a perfect proxy: increasing the default v2 prior weight from 0.45 to 0.60 improved several public-data simulator seeds, yet dropped to -0.63 on hidden Codabench.

5 Failure Modes and Lessons

The strongest failure mode was hidden-distribution mismatch. Public-data validation rewarded deeper subject-benchmark-condition priors and higher v2 prior weights, while hidden data punished over-reliance on that direction. This suggests that hidden categories contain distribution shifts where public fine-grained cells are not always the right anchor.

The second failure mode was learned residual overfitting. A Modal-trained direct residual MLP had very strong calibration performance on a held-out public slice, but hidden performance worsened as soon as the residual received substantial weight. A 15% blend tied the best score; 5%, 35%, and 55% were worse. This indicates that the residual contains some signal but is not reliable enough to carry the prediction.

The third failure mode was adaptive-label selection bias. Selecting the longest items produced revealed labels concentrated in hard coding/math categories. A global offset then applied those hard-item residuals to unrelated predictions. Per-category offsets also overfit because $K = 5$ labels are too few to estimate a stable category correction without heavy shrinkage. The final lesson is that this challenge rewards robust, boring probability estimation. The winning path for my submission was not a larger encoder or more expressive model, but a conservative prior hierarchy, defensive runtime code, stable adaptive acquisition, and clipped label-based calibration.

6 Reproducibility

The code is available at `https://github.com/paris007/torch_measure/tree/predictive-eval-challenge`. The final Codabench-ready directories live under `codabench_submissions/`; `make_submission.py` builds flat ZIP archives with `model.py` at the archive root. The best submitted artifact for archival purposes is `robust_prior_v1_submission.zip`; the best leaderboard entry is username `parism`, score -0.61, submission ID 743892.

A Iteration Log

This appendix records the broader development trail so the report reflects the full set of attempted methods, including variants that were not part of the final recommended submission. Scores are hidden Codabench negative log-loss when reported; higher is better.

Method / artifact	**Score**	**Purpose and outcome**
Embedding v1	-0.64	MiniLM item embeddings plus subject prior, class-balanced logistic head. Strong in-sample NLL/AUC did not transfer to hidden cold-start items.
Prior baseline v1	-0.64	Smoothed empirical prior. Showed that calibration and adaptive offsets alone explained much of the early performance.
Embedding v2	-0.69	Removed class balancing and added subject embedding interaction. Better smoke-test calibration, worse hidden generalization.
Factor/PGE v1	-0.61	2-PL factor model trained on 500k rows. First model with clear discriminative signal, AUC about 0.70.
Factor/PGE v2	-0.62	More rows, deeper MLP, theta regularization, and temperature calibration. Local NLL looked strong but hidden score regressed.
Factor + prior ensemble	-0.61–-0.66	Conservative logit blends tied best; per-category offsets and over-routed versions degraded sharply.
Multi-seed factor	-0.62	Modal-trained MiniLM factor ensemble. Variance reduction was real locally but too small on hidden data.
Larger encoders	-0.65	MPNet/BGE-style factor artifacts. Extra embedding capacity did not fix hidden calibration.
Robust acquisition variants	-0.61–-0.63	Ensemble, hash, offset, diverse, and multiseed variants tested acquisition and label-use choices. Hash/diverse landed near -0.62; stronger offsets fell to -0.63.
Robust prior router	-0.61–-0.63	Prior, v1, and v2 tied best at -0.61. Clipping slipped to -0.62; increasing v2 weight to 0.60 fell to -0.63.
Direct residual blends	-0.61–-0.65	Learned residual MLP had weak useful signal at 15% blend (-0.61), but 35%–55% weights overfit and worsened hidden NLL.